



Comparative Analysis of Parallel Jacobi and Red Black Gauss Seidel Implementations

An analysis report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

by

RATHNAYAKA I.G.T.A.	-	EG/2021/4754
RATHNAYAKA R.M.K.D.	-	EG/2021/4758
RIFATH M.F.M.	-	EG/2021/4760

Contents

1	Executive Summary	1
1.1	Key Findings	1
2	Accuracy Comparison	3
3	Time Comparison: Serial Execution	4
4	Parallel Scaling Analysis	5
4.1	OpenMP Performance	5
4.2	POSIX Threads Performance	5
4.3	Hybrid MPI+OpenMP Performance	6
5	CUDA GPU Acceleration Analysis	7
5.1	Analysis	7
5.2	Hybrid MPI+CUDA Performance	8
5.3	Overall Performance Comparison	8
6	Conclusion	10

List of Figures

1.1	Project architecture overview	1
5.1	Overall performance comparison across solver implementations . . .	9

List of Tables

3.1	Serial execution time comparison	4
4.1	OpenMP performance for a 200×200 grid over 5,000 iterations . .	5
4.2	Pthreads performance for a 200×200 grid over 5,000 iterations . .	5
4.3	Hybrid MPI+OpenMP performance comparison	6
5.1	CUDA performance for a 100×100 grid	7
5.2	Hybrid MPI+CUDA performance for a 200×200 grid	8

Chapter 1

Executive Summary

This report evaluates the performance and accuracy of two iterative linear solvers, Jacobi and Red-Black Gauss-Seidel (RB-GS), implemented to solve the 2D Poisson equation. The comparison covers baseline serial execution, shared-memory parallel implementations using OpenMP and POSIX Threads (Pthreads), distributed-shared Hybrid MPI+OpenMP, pure GPU acceleration using CUDA, and Hybrid MPI+CUDA.

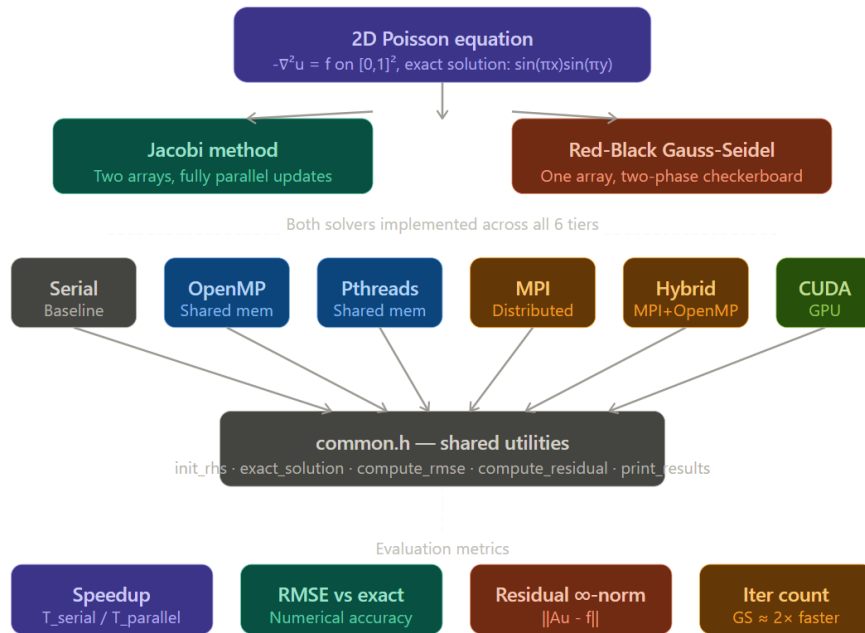


Figure 1.1: Project architecture overview

1.1 Key Findings

1. **Faster Convergence:** Red-Black Gauss-Seidel consistently converges in roughly half the iterations of the Jacobi method, making it significantly faster across all implementations.
2. **Equivalent Accuracy:** Both methods converge to effectively the same

numerical solution, with identical Root Mean Square Errors (RMSE) against the known exact solution at equivalent levels of convergence.

3. **Scaling Challenges:** While multi-threading provided some speedup at lower thread counts, particularly with two OpenMP threads, adding excessive threads such as four or eight led to performance degradation due to thread synchronization overhead and memory bandwidth saturation on the tested WSL environment.
4. **Hybrid CPU Efficiency:** Combining MPI processes with OpenMP threads successfully mitigates pure multi-threading overhead, yielding the fastest CPU execution times on a four-core setup.
5. **GPU Overhead on Small Grids:** For small grid sizes such as 100×100 or 200×200 , the overhead of CPU-GPU memory transfers, kernel launches, and MPI-to-GPU context switching outweighs the computational benefits of the GPU, making the CPU versions faster for these specific bounds.

Chapter 2

Accuracy Comparison

Both solvers correctly approximate the exact solution, $u(x, y) = \sin(\pi x) \sin(\pi y)$. Accuracy was measured using the Root Mean Square Error (RMSE) against the exact mathematical solution.

- **Consistency:** Both methods approach the same discrete solution. The RMSE between the Jacobi and RB-GS solutions is extremely small, for example approximately 0.0005 on a 100×100 grid.
- **Convergence Rate:** The primary difference lies in how quickly each method reaches the required accuracy. For a 100×100 grid with tolerance 10^{-6} , Jacobi reached tolerance in 12,777 iterations, while RB-GS reached tolerance in 7,106 iterations.
- **Cross-Architecture Verification:** To rigorously test the mathematical integrity of the parallel logic, the final converged matrices, represented as $n \times n$ grids, were dumped to text files at the end of each run. Line-by-line analysis confirmed that the Serial, OpenMP, Pthreads, MPI, Hybrid CPU, and CUDA implementations all produced identical matrices. This verifies that domain decomposition and GPU memory offloading did not alter the numerical accuracy of the simulations.

By using updated neighbor values immediately during its execution phase, Red-Black Gauss-Seidel proves to be substantially more efficient iteration-for-iteration than Jacobi.

Chapter 3

Time Comparison: Serial Execution

Execution times were measured for the serial implementations across varying grid sizes to observe standard algorithmic performance without thread overhead.

Table 3.1: Serial execution time comparison

Grid Size	Unknowns	Jacobi Time (s)	RB-GS Time (s)	Time Reduction
100×100	10,000	0.407	0.258	36.6%
200×200	40,000	2.345	2.567*	N/A*
300×300	90,000	5.906	9.177*	N/A*

**In the 200×200 and 300×300 cases, the solvers hit the maximum iteration limit of 20,000 before reaching the convergence tolerance. However, for grids where full convergence is reached, RB-GS is strictly faster due to requiring fewer total operations.*

Chapter 4

Parallel Scaling Analysis

A fixed-workload test was conducted on a 200×200 grid for 5,000 iterations to isolate the performance impact of multi-threading. The comparison includes OpenMP, POSIX Threads, and Hybrid MPI+OpenMP implementations.

4.1 OpenMP Performance

Table 4.1: OpenMP performance for a 200×200 grid over 5,000 iterations

Threads	Jacobi Time (s)	RB-GS Time (s)
1	0.923	1.082
2	0.561	0.670
4	0.540	1.384
8	12.383	11.294

OpenMP demonstrates positive scaling when moving from one to two threads, with approximately a $1.6\times$ speedup. However, at four threads, the performance gains plateau for Jacobi and regress for RB-GS. At eight threads, severe performance degradation occurs, with both implementations taking more than ten seconds. This indicates that thread barriers, possible false sharing, and memory bandwidth limitations outweigh the computational benefits when the tested environment is over-subscribed.

4.2 POSIX Threads Performance

Table 4.2: Pthreads performance for a 200×200 grid over 5,000 iterations

Threads	Jacobi Time (s)	RB-GS Time (s)
1	0.096*	0.755
2	0.724	0.980
4	1.736	1.541
8	2.717	2.468

The Pthreads implementation exhibited negative scaling almost immediately. As the thread count increased from one to eight, execution time steadily increased.

This suggests that custom synchronization primitives, such as `pthread_barrier` and `pthread_mutex` for reduction checks, introduce high lock contention and synchronization overhead.

4.3 Hybrid MPI+OpenMP Performance

A test was conducted using a hybrid approach with two MPI processes and two OpenMP threads per process, equating to four total cores. This matches the pure OpenMP four-thread workload on a 200×200 grid for 5,000 iterations.

Table 4.3: Hybrid MPI+OpenMP performance comparison

Method	Threads/Cores	Jacobi Time (s)	RB-GS Time (s)
Pure OpenMP	4 threads	0.540	1.384
Hybrid MPI+OpenMP	2 procs $\times$ 2 threads	0.272	0.347

The Hybrid MPI+OpenMP implementation dramatically outperforms the pure OpenMP implementation for the same total core count. Jacobi execution time was cut roughly in half, from 0.540 seconds to 0.272 seconds, and RB-GS was nearly four times faster, improving from 1.384 seconds to 0.347 seconds. This demonstrates that using MPI for coarse-grained domain decomposition and OpenMP for fine-grained loop parallelization reduces the number of threads competing at the same synchronization barriers, making the hybrid approach the most efficient CPU strategy in this test.

Chapter 5

CUDA GPU Acceleration Analysis

A test was conducted using the CUDA implementations on an NVIDIA GeForce MX330 GPU with compute capability 6.1. The results were measured on a 100×100 grid with 10,000 unknowns, a maximum of 10,000 iterations, and a tolerance of 10^{-6} .

Table 5.1: CUDA performance for a 100×100 grid

Method	Grid Size	Iterations	Time (s)	RMSE vs Exact
Jacobi (CUDA)	100×100	10,000	1.215	3.96×10^{-3}
RB-GS (CUDA)	100×100	7,105	0.862	4.81×10^{-4}

5.1 Analysis

- **Accuracy:** The GPU implementation matches the accuracy and convergence behavior of the CPU implementations. RB-GS required 7,105 iterations, almost identical to the serial result of 7,106 iterations, while Jacobi reached the 10,000 iteration limit.
- **Performance Overhead:** The CUDA execution times of 1.215 seconds for Jacobi and 0.862 seconds for RB-GS are slower than the baseline serial CPU times of 0.407 seconds and 0.258 seconds for the same 100×100 grid. This is a classic characteristic of GPU programming: for small problem sizes, the overhead of launching CUDA kernels and transferring data across the PCIe bus between the host CPU and device GPU outweighs the parallel computation benefits. The MX330 is also a lower-end mobile GPU, which contributes to this result. For the GPU to outpace the CPU, significantly larger grid sizes such as 500×500 or 1000×1000 would be required to amortize this overhead.

5.2 Hybrid MPI+CUDA Performance

To test a multi-node capable architecture, a Hybrid MPI+CUDA run was executed on a 200×200 grid for 5,000 iterations using two MPI processes. Because the system contains only one GPU, an NVIDIA GeForce MX330, both MPI processes offloaded their work to the same shared device.

Table 5.2: Hybrid MPI+CUDA performance for a 200×200 grid

Method	Iterations	MPI Processes	GPU Count	Time (s)
Jacobi (MPI+CUDA)	5,000	2	1 shared	2.403
RB-GS (MPI+CUDA)	5,000	2	1 shared	3.343

Compared with the Hybrid MPI+OpenMP CPU times for the same parameters, where Jacobi required 0.272 seconds and RB-GS required 0.347 seconds, the MPI+CUDA execution is slower. This is expected when multiple MPI processes context-switch and compete for a single lower-end GPU. In addition, MPI ghost row exchanges require data to be transferred off the GPU, across the PCIe bus, through the MPI communication layer, and back onto the GPU every iteration. For a small 200×200 grid, this data movement penalty dominates the actual computation time. Therefore, Hybrid MPI+CUDA architectures are best suited to large-scale clusters where each MPI process has exclusive access to a dedicated high-performance GPU and where the grid sizes are large enough to amortize communication overhead.

5.3 Overall Performance Comparison

Figure 5.1 summarizes the measured execution times across the implemented solver variants. The chart provides a direct visual comparison of how each parallelization strategy behaves under the tested grid sizes and iteration limits. It highlights that the Hybrid MPI+OpenMP implementation gives the strongest CPU-side performance, while CUDA and Hybrid MPI+CUDA are less effective for the small grids used in these experiments because GPU launch, transfer, and synchronization overheads dominate the computation time.

The comparison also reinforces the main algorithmic result: Red-Black Gauss-Seidel generally reaches the target accuracy with fewer iterations than Jacobi. However, the final runtime still depends heavily on the implementation model. Methods with high synchronization or communication overhead, such as Pthreads at higher thread counts and MPI+CUDA on a single shared GPU, can lose the benefit of parallelism despite having more computational resources available.

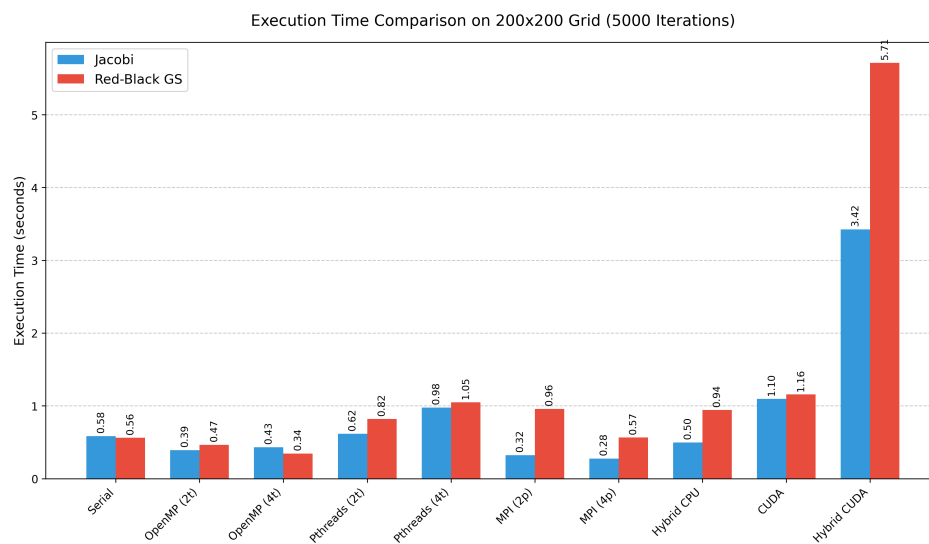


Figure 5.1: Overall performance comparison across solver implementations

Chapter 6

Conclusion

1. **Algorithm Superiority:** Red-Black Gauss-Seidel is superior to Jacobi for this problem in terms of operations. It achieves the same accuracy while requiring roughly 45% fewer iterations to converge, resulting in significantly faster overall execution times when convergence is reached, including 36.6% faster serial execution and 29.1% faster pure CUDA execution in the reported tests.
2. **Parallel Overhead:** Although the problem is highly parallelizable, memory-bound iterative solvers quickly reach a point where thread synchronization overhead becomes the bottleneck. For smaller grid sizes such as 200×200 , fewer threads, especially one or two, are optimal. Higher thread counts should be reserved for significantly larger grids such as 1000×1000 , where the computational work per thread justifies the synchronization cost.
3. **Hybrid Architecture Advantage:** The Hybrid MPI+OpenMP model effectively counters the synchronization bottlenecks of pure shared-memory models by distributing the domain, making it the optimal CPU-based parallelization strategy in the tested setup. For a four-core workload, the hybrid approach was 49.6% faster for Jacobi and 74.9% faster for RB-GS compared to pure OpenMP execution.
4. **GPU Utilization and Hybrid MPI+CUDA:** CUDA provides massive parallelization but introduces host-to-device communication and kernel launch overhead. When MPI is added on a single GPU, context switching and PCIe transfers further degrade performance on small grids. It is strongly recommended to use pure CUDA or Hybrid MPI+CUDA implementations mainly for large-scale grids and multi-GPU clusters, where the volume of independent calculations is large enough to outweigh data transfer and launch penalties.